Reviewer Pack — Minimal Rank of Commutative Algebraic Curves vs AC0 Parity C…

Entry c966aae7ce50 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 12:38:04 UTC

Minimal Rank of Commutative Algebraic Curves vs AC0 Parity Circuit Size

Entry ID: `c966aae7ce50` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 12:38:04 UTC

1. Conjecture

Field A (mathematical branch): Commutative Algebra (Algebraic Curves) **Field B** (complexity object): Complexity Theory: AC0 Parity Complexity

Statement:

> ['For every Boolean function f computable by an AC0 parity circuit of size s, there exists a commutative algebraic curve with genus g such that the rank of the algebraic variety defined by its defining equations is R(f) = Θ(g^2 / s)', 'where g is the smallest integer satisfying this inequality for a given function f.']

Rationale (proposer's reasoning):

> ['Algebraic curves have been used in various contexts to encode complexity-theoretic information, such as in the study of counting complexity. The proposed conjecture aims to leverage the algebraic structure of curves to provide new insights into the AC0 parity circuit size.', 'This bridge might expose a deeper connection between algebraic geometry and computational complexity, potentially leading to new algorithms or lower bounds.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8c6b9e294e0ac02d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture regarding commutative algebraic curves and AC0 parity circuit size, support is indicated by an observed correlation coefficient r >= 0.9 between R(f) and s for all functions, with a slope within ±10% of g^2 / s.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "commutative algebra" AND "algebraic curve" AND AC0 parity circuit" - "AC0 parity complexity" AND minimal rank" AND commutative algebra" - "genus g" AND ".\. R(f) = 0(g^2 / s)" AND AC0 parity circuit size

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    rank = 0
    for i in range(n):
        if A[i][rank] == 0:
            swap_found = False
            for k in range(i+1, n):
                if A[k][rank] != 0:
                    A[i], A[k] = A[k], A[i]
                    swap_found = True
                    break
            if not swap_found:
                continue
        factor = Fraction(A[i][rank], A[rank][rank])
        for k in range(rank, n):
            A[i][k] -= factor * A[rank][k]
        rank += 1
    return rank

def characteristic_polynomial(truth_table):
    n = len(truth_table)
    poly = [0] * (n + 1)
    poly[0] = 1
    for i in range(1, n + 1):
        poly[i % n] += truth_table[i - 1]
    return poly

def rank_of_variety(poly, n):
    A = []
```

```
    for i in range(n):
        row = [poly[j] * (-1)**(j & (1 << i)) for j in range(n)]
        A.append(row)
    return gaussian_elimination(A)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        s = random.randint(1, n)
        truth_table = [random.choice([0, 1]) for _ in range(n)]
        poly = characteristic_polynomial(truth_table)
        rank = rank_of_variety(poly, n)
        g = math.ceil(math.sqrt(s))

        results.append({
            "n": n,
            "s": s,
            "g": g,
            "rank": rank
        })

    if not results:
        return {
            "metric_name": "R(f)",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "empty_results"
        }

    R_values = [result["rank"] for result in results]
    s_values = [result["s"] for result in results]
    g_values = [result["g"] for result in results]

    mean_R = sum(R_values) / len(R_values)
    std_R = math.sqrt(sum((x - mean_R)**2 for x in R_values) / len(R_values))
    slope = (mean_R * s_values[0]) / g_values[0]**2

    correlation_coefficient = sum((R_values[i] - mean_R) * (s_values[i] - mean(s_values)) for i in

    conjecture_holds = correlation_coefficient >= 0.9 and abs(slope - g_values[0]**2 / s_values[0]

    return {
        "metric_name": "R(f)",
        "metric_value": mean_R,
        "instances_tested": len(R_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"correlation_coefficient={correlation_coeff
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
```

```
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")

    results = [run_trial(seed) for seed in seeds]
    R_values = [result["metric_value"] for result in results if result["instances_tested"] > 0]
    support_fraction = sum(result["conjecture_holds"] for result in results if result["instances_t

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={sum(R_values)/len(R_values)} std={math.sqrt(sum((x - sum(R
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7d87138.py", line 105, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7d87138.py", line 88, in run_trial
    correlation_coefficient = sum((R_values[i] - mean_R) * (s_values[i] - mean(s_values)) for i in rang
mean(s_values))**2 for x in s_values)))
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7d87138.py", line 88, in <genexpr>
    correlation_coefficient = sum((R_values[i] - mean_R) * (s_values[i] - mean(s_values)) for i in rang
mean(s_values))**2 for x in s_values)))
                                                                                ^^^^

NameError: name 'mean' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means that the pre-registered support condition could not be evaluated. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12695
2	propose	ollama_remote	glm4:latest	0	0	11184
3	preregistration	ollama_remote	glm4:latest	0	0	5603
4	novelty	ollama_remote	glm4:latest	0	0	4845
5	novelty	ollama_remote	glm4:latest	0	0	6050
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13540
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14276
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12633
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13171
10	judge	ollama_remote	glm4:latest	0	0	8780

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 102777 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/c966aae7ce50.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c966aae7ce50.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c966aae7ce50.tar.gz` (if generated)